Data cleaning and manipulation are critical steps in the data preprocessing pipeline for any data analysis or machine learning project. These processes involve preparing raw data for analysis by identifying and addressing errors, inconsistencies, missing values, and other issues. Here are some common techniques and tools used in data cleaning and manipulation:

A. Data Cleaning Techniques

1. Handling Missing Values

- Deletion: Remove rows or columns with missing values.

Step by step deletion of missing values:

Step 1: Import the pandas for loading, cleaning, and manipulation of the datasets.

```
#Import neede libraries
import pandas as pd
```

Step 2: Load and display the datasets.

```
#Load the datasets
df = pd.read_csv("titanic.csv")
print(df.head())
✓ 0.0s

   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

Step 3: Drop data that we won't be working.

```
#Let's drop data that we won't be working.
df.drop("Name", axis=1, inplace=True)
df.drop("Ticket", axis=1, inplace=True)
df.drop("PassengerId", axis=1, inplace=True)
df.drop("Cabin", axis=1, inplace=True)
df.drop("Embarked", axis=1, inplace=True)
```

This is now what the data looks like:

```
print(df.head())
✓ 0.0s

   Survived  Pclass     Sex   Age  SibSp  Parch     Fare
0         0       3    male  22.0      1      0   7.2500
1         1       1  female  38.0      1      0  71.2833
2         1       3  female  26.0      0      0   7.9250
3         1       1  female  35.0      1      0  53.1000
4         0       3    male  35.0      0      0   8.0500
```

Step 4: Since the sex or gender is in string value let's convert it into numerical value. Let's import first LabelEncoder from sklearn.

```
#Converting string value to numerical value.
#Import first this library
from sklearn.preprocessing import LabelEncoder
```

Step 5: Convert the string value of column sex into numerical value using LabelEncoder.

```
le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex'])
newdf = df
```

Step 6: Print the new datasets to see the changes.

```
print(newdf.head())
✓ 0.0s

   Survived  Pclass  Sex   Age  SibSp  Parch     Fare
0         0       3    1  22.0      1      0   7.2500
1         1       1    0  38.0      1      0  71.2833
2         1       3    0  26.0      0      0   7.9250
3         1       1    0  35.0      1      0  53.1000
4         0       3    1  35.0      0      0   8.0500
```

Step 7: Checking the datasets if there are missing values.

```
#Check if there is missing values
newdf.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Survived  891 non-null    int64
 1   Pclass    891 non-null    int64
 2   Sex       891 non-null    int32
 3   Age       714 non-null    float64
 4   SibSp     891 non-null    int64
 5   Parch     891 non-null    int64
 6   Fare      891 non-null    float64
dtypes: float64(2), int32(1), int64(4)
memory usage: 45.4 KB
```

The age has 714 non-null values and the other are 891 non-null value, that's means there are 177 missing values.

Step 8: Another way to check is there are missing values using isnull function and sum function.

```
#another way to check if there are missing values.
print(newdf.isnull().sum())
```

✓ 0.0s

```
Survived      0
Pclass        0
Sex           0
Age         177
SibSp         0
Parch         0
Fare          0
dtype: int64
```

The age has total of 177 missing values.

Step 9: Now, delete the column with the missing values.

```
#Deleting the column with missing data.
updated_df = newdf.dropna(axis=1)
print(updated_df.isnull().sum())
```

✓ 0.0s

```
Survived    0
Pclass      0
Sex         0
SibSp       0
Parch       0
Fare        0
dtype: int64
```

When we check again, there are no missing value.

Step 10: You can also delete missing value by rows, just change the axis value in 0.

```
#Deleting the row with missing data.
updated_df_2 = newdf.dropna(axis=0)
```

- **Imputation**: Fill in missing values with suitable replacements (e.g., mean, median, mode, or more advanced methods like regression imputation).

Step by step imputation of missing values:

Step 1: I just continue to use the datasets. Since I already have import the needed library and drop the data that I won't be using. Let's fill the missing value with the mean or median value.

```
#Filling the missing value or data with the mean or median value.
updated_df = newdf
updated_df['Age']=updated_df['Age'].fillna(updated_df['Age'].mean())
updated_df.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Survived  891 non-null    int64
 1   Pclass    891 non-null    int64
 2   Sex       891 non-null    int32
 3   Age       891 non-null    float64
 4   SibSp     891 non-null    int64
 5   Parch     891 non-null    int64
 6   Fare      891 non-null    float64
dtypes: float64(2), int32(1), int64(4)
memory usage: 45.4 KB
```

Step 2: Another way to impute is imputation of another column. We are going to add column called Ageismissing to know if it is null value or not null value.

```
#Imputation with an additional column.
updated_df = newdf
updated_df['Ageismissing'] = updated_df['Age'].isnull()
```

Step 3: Let's import first the SimpleImputer from sklearn library.

```
#import the needed libraries.
from sklearn.impute import SimpleImputer
```

Step 4: Impute the missing value using the median.

```
#selects all column that contain numerical values
numeric_cols = updated_df.select_dtypes(include='number').columns

my_imputer = SimpleImputer(strategy= 'median')

#imputing missing values by using median
updated_df[numeric_cols] = my_imputer.fit_transform(updated_df[numeric_cols])


print(updated_df.isnull().sum())
updated_df.info()
```

✓ 0.0s

```
Survived      0
Pclass        0
Sex           0
Age           0
SibSp         0
Parch         0
Fare          0
MissingAge    0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Survived    891 non-null    float64
 1   Pclass      891 non-null    float64
 2   Sex         891 non-null    float64
 3   Age         891 non-null    float64
 4   SibSp       891 non-null    float64
 5   Parch       891 non-null    float64
 6   Fare        891 non-null    float64
 7   MissingAge  891 non-null    bool
dtypes: bool(1), float64(7)
memory usage: 49.7 KB
```

As you can see there is new column "Ageismissing". If we display the output dataset:

```
print(updated_df.head())
✓ 0.0s

   Survived  Pclass  Sex   Age  SibSp  Parch     Fare  Ageismissing
0         0       3    1  22.0      1      0   7.2500         False
1         1       1    0  38.0      1      0  71.2833         False
2         1       3    0  26.0      0      0   7.9250         False
3         1       1    0  35.0      1      0  53.1000         False
4         0       3    1  35.0      0      0   8.0500         False
```

It will shows that the Ageismissing has now boolean value.

2. **Data Type Conversion**:

- Ensure that data types are consistent and appropriate for analysis.
- Convert categorical variables to numerical format.

Step By Step Data Type Conversion:

Step 1: Import the needed library.

```
#import the needed library
import pandas as pd
✓ 0.0s
```

Step 2: Load the datasets.

```
#Load the datasets
df = pd.read_csv('titanic.csv')
print(df.head())
✓ 0.1s

   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0
```

Step 3: Let's check what are the data types in our datasets.

```
print(df.dtypes)
✓ 0.0s

PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

Step 4: Convert the 'Sex' column from object to integer using mapping dictionary. Let's create first a mapping dictionary.

```
#Convert the Sex column from object to integer.
#Create a mapping disctionary for 'Sex' column
sex_mapping = {'male': 0, 'female': 1}
✓ 0.0s
```

Step 5: Apply the mapping to convert the 'Sex' column.

```
#Apply the mapping to convert the 'Sex' column.
df['Sex'] = df['Sex'].map(sex_mapping)
print(df.head())
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name  Sex   Age  SibSp  Parch  \
0                            Braund, Mr. Owen Harris    0  22.0      1      0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...    1  38.0      1      0
2                             Heikkinen, Miss. Laina    1  26.0      0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)    1  35.0      1      0
4                           Allen, Mr. William Henry    0  35.0      0      0

            Ticket     Fare Cabin Embarked
0        A/5 21171   7.2500   NaN        S
1         PC 17599  71.2833   C85        C
2 STON/O2. 3101282   7.9250   NaN        S
3           113803  53.1000  C123        S
4           373450   8.0500   NaN        S
```

Step 6: There is another way to convert the object to integer. Its Label Encoder from sklearn. Let's import first the needed library.

```
#import needed library
from sklearn.preprocessing import LabelEncoder
```

Step 7: Load the datasets.

```
#load the datasets.
df1 = pd.read_csv('titanic.csv')
print(df1.head())
✓ 0.0s

   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

Step 8: Convert the object to integer.

```
#convert the object to integer.
le = LabelEncoder()
df1['Sex'] = le.fit_transform(df1['Sex'])
newdf1 = df1
✓ 0.0s

print(newdf1.head())
✓ 0.0s

   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name  Sex   Age  SibSp  Parch  \
0                            Braund, Mr. Owen Harris    1  22.0      1      0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...    0  38.0      1      0
2                             Heikkinen, Miss. Laina    0  26.0      0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)    0  35.0      1      0
4                           Allen, Mr. William Henry    1  35.0      0      0
```

Step 9: Now let's convert float to integer using astype. For this tutorial we are continue to use the newdf1.

```
#Convert float to integer using astype.
newdf1['Fare'] = newdf1['Fare'].astype(int)
print(newdf1.head())
✓ 0.0s

   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name  Sex   Age  SibSp  Parch  \
0                                Braund, Mr. Owen Harris    1  22.0      1      0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...    0  38.0      1      0
2                                 Heikkinen, Miss. Laina    0  26.0      0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)    0  35.0      1      0
4                               Allen, Mr. William Henry    1  35.0      0      0

             Ticket  Fare Cabin Embarked
0         A/5 21171     7   NaN        S
1          PC 17599    71   C85        C
2  STON/O2. 3101282     7   NaN        S
3            113803    53  C123        S
4            373450     8   NaN        S
```

3. Handling Duplicates

- Identify and remove duplicate records from the dataset.

Step by Step Tutorial:

Step 1: Import the needed library.

```
#import needed library
import pandas as pd
```

Step 2: Load the dataset.

```
#load dataset
df = pd.read_csv('steam-200k.csv', names=['id', 'video_games', 'behavior', 'hours', 'integer'])
df
✓ 0.3s
```

	id	video_games	behavior	hours	integer
0	151603712	The Elder Scrolls V Skyrim	purchase	1.0	0
1	151603712	The Elder Scrolls V Skyrim	play	273.0	0
2	151603712	Fallout 4	purchase	1.0	0
3	151603712	Fallout 4	play	87.0	0
4	151603712	Spore	purchase	1.0	0
...	...	...	...	...	...
199995	128470551	Titan Souls	play	1.5	0
199996	128470551	Grand Theft Auto Vice City	purchase	1.0	0
199997	128470551	Grand Theft Auto Vice City	play	1.5	0
199998	128470551	RUSH	purchase	1.0	0
199999	128470551	RUSH	play	1.4	0

200000 rows × 5 columns

Step 3: Check if there are duplicates of data.

```
#check if there are duplicates of data
duplicates = df.duplicated()
df[duplicates]
```

✓ 0.1s

	id	video_games	behavior	hours	integer
1968	11373749	Sid Meier's Civilization IV	purchase	1.0	0
1970	11373749	Sid Meier's Civilization IV Beyond the Sword	purchase	1.0	0
1972	11373749	Sid Meier's Civilization IV Warlords	purchase	1.0	0
2724	56038151	Grand Theft Auto San Andreas	purchase	1.0	0
2726	56038151	Grand Theft Auto Vice City	purchase	1.0	0
...	...	...	...	...	...
194392	39146470	Sid Meier's Civilization IV Warlords	purchase	1.0	0
195517	48666962	Crysis 2	purchase	1.0	0
195821	112845094	Grand Theft Auto San Andreas	purchase	1.0	0
195823	112845094	Grand Theft Auto Vice City	purchase	1.0	0
195824	112845094	Grand Theft Auto III	purchase	1.0	0

707 rows × 5 columns

Step 4: Drop the duplicates data.

```
#drop duplicate data
cleandf = df.drop_duplicates()
cleandf
```

✓ 0.1s

	id	video_games	behavior	hours	integer
0	151603712	The Elder Scrolls V Skyrim	purchase	1.0	0
1	151603712	The Elder Scrolls V Skyrim	play	273.0	0
2	151603712	Fallout 4	purchase	1.0	0
3	151603712	Fallout 4	play	87.0	0
4	151603712	Spore	purchase	1.0	0
...	...	...	...	...	...
199995	128470551	Titan Souls	play	1.5	0
199996	128470551	Grand Theft Auto Vice City	purchase	1.0	0
199997	128470551	Grand Theft Auto Vice City	play	1.5	0
199998	128470551	RUSH	purchase	1.0	0
199999	128470551	RUSH	play	1.4	0

199293 rows × 5 columns

Step 5: If you don't want to drop all the duplicates and keep the first occurance of data, just add the 'keep' parameter.

```
#keeping the first occurence of the duplicates
cleandf2 = df.drop_duplicates(keep='first')
cleandf2
```

✓ 0.1s

	id	video_games	behavior	hours	integer
0	151603712	The Elder Scrolls V Skyrim	purchase	1.0	0
1	151603712	The Elder Scrolls V Skyrim	play	273.0	0
2	151603712	Fallout 4	purchase	1.0	0
3	151603712	Fallout 4	play	87.0	0
4	151603712	Spore	purchase	1.0	0
...	...	...	...	...	...
199995	128470551	Titan Souls	play	1.5	0
199996	128470551	Grand Theft Auto Vice City	purchase	1.0	0
199997	128470551	Grand Theft Auto Vice City	play	1.5	0
199998	128470551	RUSH	purchase	1.0	0
199999	128470551	RUSH	play	1.4	0

199293 rows × 5 columns

Step 6: If you don't want to delete or drop everything you can just replace the duplicate values with median of the column.

```
# Replace duplicate values with the median of the column
df['id'] = df.groupby('video_games')['id'].transform('median')
df
```

✓ 0.0s

	id	video_games	behavior	hours	integer
0	96696864.0	The Elder Scrolls V Skyrim	purchase	1.0	0
1	96696864.0	The Elder Scrolls V Skyrim	play	273.0	0
2	71527252.0	Fallout 4	purchase	1.0	0
3	71527252.0	Fallout 4	play	87.0	0
4	111754806.0	Spore	purchase	1.0	0
...	...	...	...	...	...
199995	64787956.0	Titan Souls	play	1.5	0
199996	71411882.0	Grand Theft Auto Vice City	purchase	1.0	0
199997	71411882.0	Grand Theft Auto Vice City	play	1.5	0
199998	78560022.0	RUSH	purchase	1.0	0
199999	78560022.0	RUSH	play	1.4	0

200000 rows × 5 columns

4. Text Cleaning

- Remove special characters, white spaces, and punctuation from text data.
- Convert text to lowercase.

Step by Step Tutorial

Step 1: Import needed library

```
#import needed library
import pandas as pd
import string
import re
```

Step 2: Load datasets.

```
#load datasets
df = pd.read_csv('titanic.csv')
df
```

✓ 0.2s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 12 columns

Step 3: Converting all text to lowercase.

```
#Lowercasing
df['Name'] = df['Name'].str.lower()
df
```

✓ 0.0s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	braund, mr. owen harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	cumings, mrs. john bradley (florence briggs th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	heikkinen, miss. laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	futrelle, mrs. jacques heath (lily may peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	allen, mr. william henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	montvila, rev. juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	graham, miss. margaret edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	johnston, miss. catherine helen "carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	behr, mr. karl howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	dooley, mr. patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 12 columns

Step 4: Or converting them to uppercase.

```
#Uppercasing
df['Name'] = df['Name'].str.upper()
df
```

✓ 0.0s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	BRAUND, MR. OWEN HARRIS	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	CUMINGS, MRS. JOHN BRADLEY (FLORENCE BRIGGS TH...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	HEIKKINEN, MISS. LAINA	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	FUTRELLE, MRS. JACQUES HEATH (LILY MAY PEEL)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	ALLEN, MR. WILLIAM HENRY	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	MONTVILA, REV. JUOZAS	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	GRAHAM, MISS. MARGARET EDITH	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	JOHNSTON, MISS. CATHERINE HELEN "CARRIE"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	BEHR, MR. KARL HOWELL	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	DOOLEY, MR. PATRICK	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 12 columns

Step 5: Removing special characters like comma, period, etc.

```
#Removing special characters
df['Name'] = df['Name'].apply(lambda x: re.sub(r'[^a-zA-Z\s]', '', x))
df
```

✓ 0.0s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	BRAUND MR OWEN HARRIS	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	CUMINGS MRS JOHN BRADLEY FLORENCE BRIGGS THAYER	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	HEIKKINEN MISS LAINA	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	FUTRELLE MRS JACQUES HEATH LILY MAY PEEL	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	ALLEN MR WILLIAM HENRY	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	MONTVILA REV JUOZAS	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	GRAHAM MISS MARGARET EDITH	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	JOHNSTON MISS CATHERINE HELEN CARRIE	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	BEHR MR KARL HOWELL	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	DOOLEY MR PATRICK	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 12 columns

Step 6: Removing numbers.

```
#Removing numbers
df['Ticket'] = df['Ticket'].apply(lambda x: re.sub(r'\d+', '', x))
df
```

✓ 0.1s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	BRAUND MR OWEN HARRIS	male	22.0	1	0	A/	7.2500	NaN	S
1	2	1	1	CUMINGS MRS JOHN BRADLEY FLORENCE BRIGGS THAYER	female	38.0	1	0	PC	71.2833	C85	C
2	3	1	3	HEIKKINEN MISS LAINA	female	26.0	0	0	STON/O.	7.9250	NaN	S
3	4	1	1	FUTRELLE MRS JACQUES HEATH LILY MAY PEEL	female	35.0	1	0		53.1000	C123	S
4	5	0	3	ALLEN MR WILLIAM HENRY	male	35.0	0	0		8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	MONTVILA REV JUOZAS	male	27.0	0	0		13.0000	NaN	S
887	888	1	1	GRAHAM MISS MARGARET EDITH	female	19.0	0	0		30.0000	B42	S
888	889	0	3	JOHNSTON MISS CATHERINE HELEN CARRIE	female	NaN	1	2	W./C.	23.4500	NaN	S
889	890	1	1	BEHR MR KARL HOWELL	male	26.0	0	0		30.0000	C148	C
890	891	0	3	DOOLEY MR PATRICK	male	32.0	0	0		7.7500	NaN	Q

891 rows × 12 columns

Step 7: Those I demonstrate are one of the Text cleaning techniques, but there are other text cleaning like:

Removing Extra Whitespaces - Remove extra whitespace and replace it with a single space.

```
#Removing extra whitespaces.
df['column_name'] = df['column_name'].apply(lambda x: ' '.join(x.split()))
df
```

Removing Punctuation- Remove any punctuation marks from the text, as they often don't contribute much to the analysis.

```
#Removing puctuation marks
df['Name'] = df['Name'].apply(lambda x: re.sub(r'[^\w\s]', '', x))
df
```

5. **Date and Time Formatting**

- Standardize date and time formats.
- Extract meaningful features from date and time data (e.g., day of the week, month, year).

Step by Step Coding:

Step 1: Import needed library.

```
#import needed library
import pandas as pd
import datetime
```

Step 2: Load the datasets.

```
#Load datasets.
data = pd.read_csv('flight_schedule.csv')
data.head()
```

✓ 0.1s

	flight_number	departure_airport	arrival_airport	departure_datetime	arrival_datetime
0	AA101	LAX	JFK	2023-09-01 08:00:00	2023-09-01 16:30:00
1	UA205	SFO	ORD	2023-09-02 10:30:00	2023-09-02 18:15:00
2	DL301	ATL	MIA	2023-09-03 14:45:00	2023-09-03 16:30:00
3	WN415	DEN	SEA	2023-09-04 12:15:00	2023-09-04 14:00:00
4	BA512	LHR	CDG	2023-09-05 16:30:00	2023-09-05 18:00:00

Step 3: Convert datetime columns to datetime format.

```
# Convert datetime columns to datetime format
data['departure_datetime'] = pd.to_datetime(df['departure_datetime'])
data['arrival_datetime'] = pd.to_datetime(df['arrival_datetime'])
```

Before converting:

```
print(data.dtypes)
✓ 0.0s

flight_number          object
departure_airport      object
arrival_airport        object
departure_datetime     object
arrival_datetime       object
dtype: object
```

After converting:

```
print(data.dtypes)
✓ 0.0s

flight_number                  object
departure_airport              object
arrival_airport                object
departure_datetime    datetime64[ns]
arrival_datetime      datetime64[ns]
dtype: object
```

Step 4: Extract components from datetime columns.

```
# Extract components from datetime columns
data['departure_year'] = data['departure_datetime'].dt.year
data['departure_month'] = data['departure_datetime'].dt.month
data['departure_day'] = data['departure_datetime'].dt.day
data['departure_hour'] = data['departure_datetime'].dt.hour
data['departure_minute'] = data['departure_datetime'].dt.minute
✓ 0.1s

data
✓ 0.0s
```

	flight_number	departure_airport	arrival_airport	departure_datetime	arrival_datetime	departure_year	departure_month	departure_day	departure_hour	departure_minute
0	AA101	LAX	JFK	2023-09-01 08:00:00	2023-09-01 16:30:00	2023	9	1	8	0
1	UA205	SFO	ORD	2023-09-02 10:30:00	2023-09-02 18:15:00	2023	9	2	10	30
2	DL301	ATL	MIA	2023-09-03 14:45:00	2023-09-03 16:30:00	2023	9	3	14	45
3	WN415	DEN	SEA	2023-09-04 12:15:00	2023-09-04 14:00:00	2023	9	4	12	15
4	BA512	LHR	CDG	2023-09-05 16:30:00	2023-09-05 18:00:00	2023	9	5	16	30

Step 5: Calculate duration of flights.

```
# Calculate duration of flights
data['flight_duration'] = data['arrival_datetime'] - data['departure_datetime']
data
✓ 0.0s
```

	flight_number	departure_airport	arrival_airport	departure_datetime	arrival_datetime	departure_year	departure_month	departure_day	departure_hour	departure_minute	flight_duration
0	AA101	LAX	JFK	2023-09-01 08:00:00	2023-09-01 16:30:00	2023	9	1	8	0	0 days 08:30:00
1	UA205	SFO	ORD	2023-09-02 10:30:00	2023-09-02 18:15:00	2023	9	2	10	30	0 days 07:45:00
2	DL301	ATL	MIA	2023-09-03 14:45:00	2023-09-03 16:30:00	2023	9	3	14	45	0 days 01:45:00
3	WN415	DEN	SEA	2023-09-04 12:15:00	2023-09-04 14:00:00	2023	9	4	12	15	0 days 01:45:00
4	BA512	LHR	CDG	2023-09-05 16:30:00	2023-09-05 18:00:00	2023	9	5	16	30	0 days 01:30:00

Activity

Compare Excel and Python as tools for data cleaning based on various criteria and the following questions.

Use the table below to compare Excel and Python:

Criteria	Excel	Python
Scalability (How well does it handle large datasets?)		
Functionality (What data cleaning tasks can it perform?)		
Ease of Use (Is it user-friendly?)		
Automation (Can repetitive tasks be automated?)		
Flexibility (Can it handle different types of data easily?)		
Performance (How fast does it process data?)		
Visualization (How well does it help in understanding data?)		
Collaboration (How easy is it to share and work with others?)		

1. Based on your findings, which tool is better for small datasets? Why?
2. Which tool is more suitable for handling large datasets? Explain.
3. How does automation differ between Excel and Python?
4. Which tool is more accessible for beginners? Why?
5. In what scenarios would you prefer Excel over Python, and vice versa?

Note: Save this activity on a separate word file with the filename SURNAME-DATA-CLEANING COMPARISON .